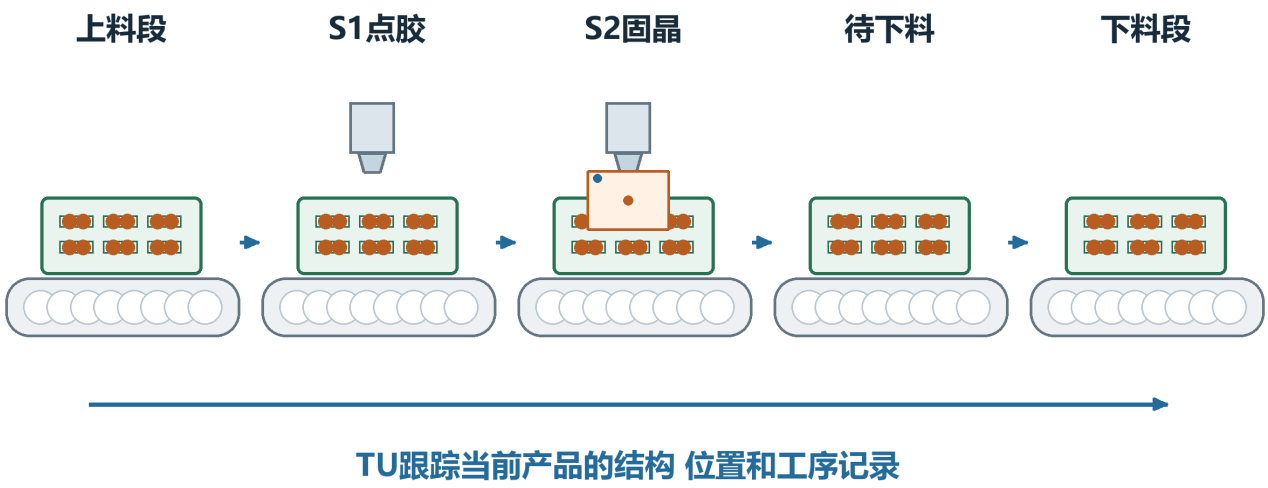
Transport Unit 设计文档

跟着一块产品走过点胶和固晶工位，看懂软件怎样记住产品、找到焊点、保存加工进度。



从第一页开始顺着看，每一页只讲一件事。先看设备在做什么，再看图中的记录或位置怎样改变。

TU 册一直跟踪“基板 1、基岛 2、A 点”；纠偏册用独立数字例子逐步算。图中布局为示意，例子数值用于理解，不是本机参数。

按设备流程阅读

先读完整流程，再按遇到的问题回看对应页。

01 先跟着产品走一遍 3
02 先认准这一块产品上的 A 点 4
03 配方和当前产品各管什么 5
04 为什么第四个点不一定第四个加工 6
05 来料后软件怎样建立加工记录 7
06 设备怎么知道 A 点在哪里 8
07 产品转了方向 A 点会到哪里 9
08 点胶工位还有一份整体偏移 10
09 拍照定位和测高改了什么 11
10 A 点点完胶后记录写到哪里 12
11 点胶完成后怎样交给固晶工位 13
12 为什么点过胶却还需要重新定位 14
13 到固晶工位一定会新建 TU 吗 15
14 固晶完成后为什么有两套完成记录 16
15 产品做完以后怎样离开设备 17
16 分两段点胶时产品记录怎么跟着走 18
17 停用一个点和加工完成有什么区别 19
18 在生产 Mapping 里点正常或成功会怎样 20
19 配方 Mapping 改的是后续产品的规则 21
20 搜索物料会不会找回原来的进度 22
21 清空载具到底清了哪些东西 23
22 换了数量或产品形状为什么要重建 24
23 异形编辑中的名字相同为什么还找不到 25
24 导入 Mapping 为什么不能等于原样续作 26
25 保存过配置重启就能继续吗 27
26 交接报错时产品可能已经走了一半 28
27 示教和测试用的 TU 是哪一份 29
28 开发时按这个顺序查到真正的 A 点 30
29 把同一个 A 点从来料追到完成 31

01 先跟着产品走一遍

先把设备想成几个接力工作的工位。一块产品送进来，点胶侧在指定位置点胶，固晶侧再把芯片贴到对应位置，最后送往下料。两侧可以各忙各的产品，靠来料和放行信号配合。

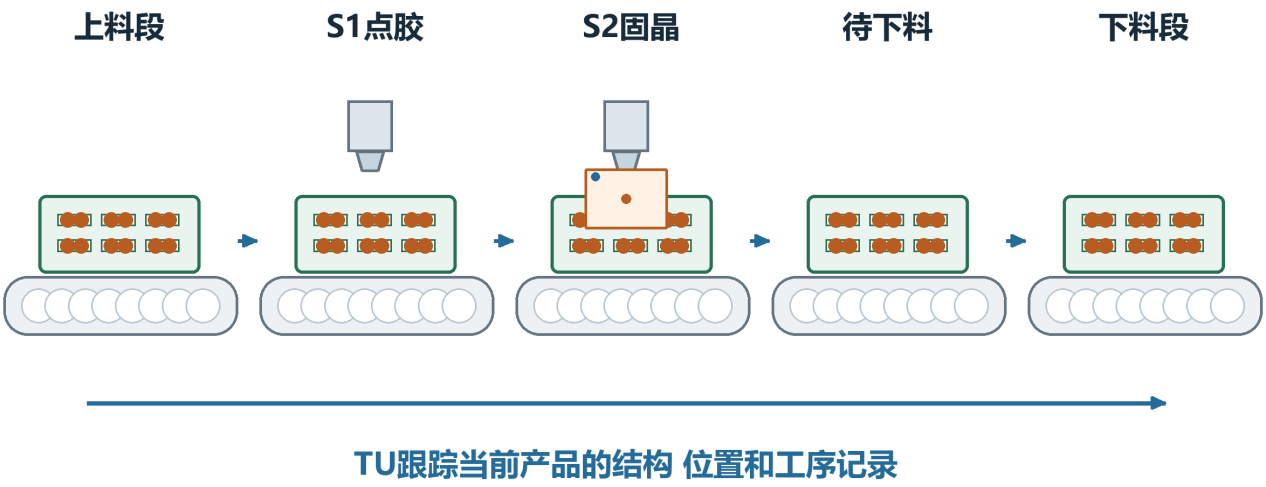


图 1 从左向右看产品路线 再看下方的软件记录

本文把点胶侧叫 S1，固晶侧叫 S2。上料、点胶、固晶、待下料和下料是输送区段。图里只画理解 TU 所需的动作，具体设备可按配方启用、跳过或细分工序。

产品换了工位，软件还得认得它：有哪些加工位置，哪个点点过胶，哪个点还没有贴芯片。这份跟着产品走的数据，叫 Transport Unit，后面简称 TU。可以先把它理解成这块产品的电子加工记录。

TU 提供产品数据；皮带、气缸和焊头的实际运动由各自控制模块执行。

02 先认准这一块产品上的 A 点

本册一直用同一组小产品举例：一份 TU 里有 2 块基板，每块基板有 3 个基岛，每个基岛有 A、B 两个加工位置。这里的“焊点”就是软件给加工位置起的名字，不一定表示要焊接。

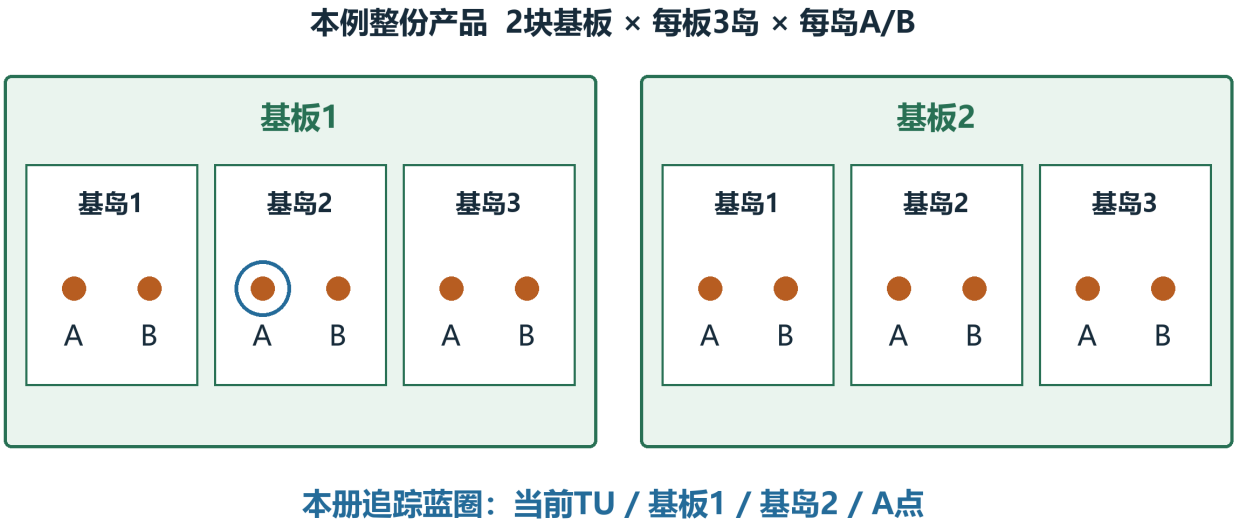


图 2 圈出的 A 点属于基板 1 的基岛 2

整份产品共有 2×3×2 = 12 个焊点。每个岛都有 A 点，所以只说“去 A 点”不够；完整说法是“当前产品，基板 1，基岛 2，A 点”。后面看到蓝圈时，跟踪的都是这个点。

软件按 TU、基板、基岛、焊点四层保存对象。基板和基岛帮忙组织位置；具体点胶、固晶是否完成，记在各个焊点自己的记录里。产品实物怎样分区要看配方，图形只表达本例层级。



先确定是哪块产品、哪块基板、哪个岛，再看焊点名。

03 配方和当前产品各管什么

配方像同一款产品的加工说明：有多少块基板、A 点在哪、要做哪些工序。TU 记的是眼前这一份产品：当前坐标是什么、A 点有没有完成点胶。动作控制器另存本轮接下来要执行的动作。

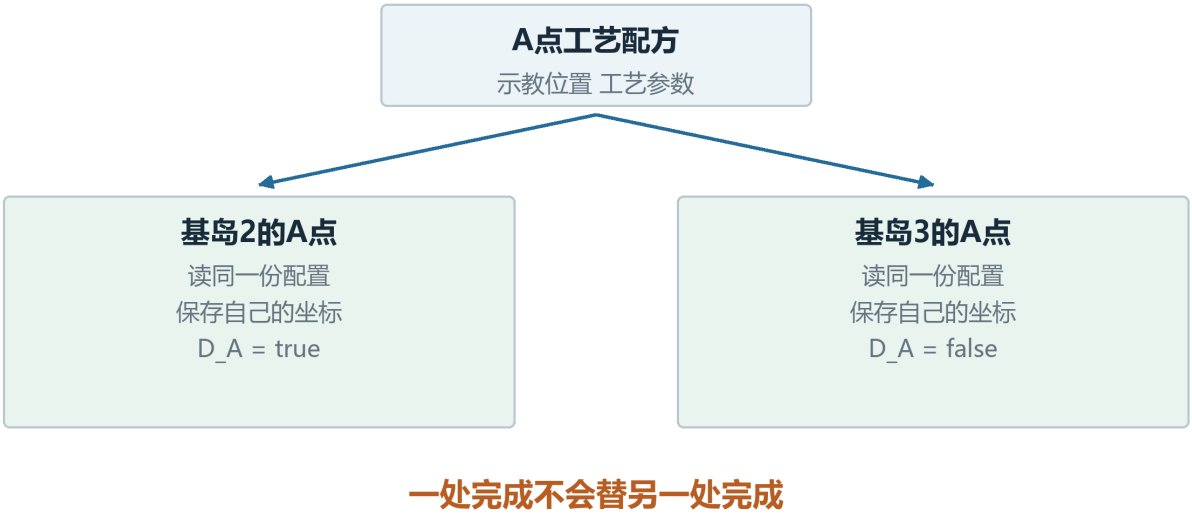


图 3 一份配方可以供多个 A 点读取 进度分别保管

基岛 2 的 A 点和基岛 3 的 A 点，可以读取同一个 A 点配置，但有各自的运行记录。基岛 2 点完胶，只把自己的点胶工序改成已完成，基岛 3 仍待加工。

常规 TU 读取配置时，会到当前生效的配方里取。它没有把整份创建时配方都锁在自己内部。因此换了配方，旧 TU 仍存在，也不表示它已经和新配方匹配。

配方说应该怎么做；TU 说这份产品做到哪；动作控制器说本轮走到哪一步。

04 为什么第四个点不一定第四个加工

规则阵列里有两种编号：结构序号 Index 方便找到对象，工作序号 WorkIndex 决定加工先后。平时顺着加工，两者常常一样；改成蛇形顺序，就容易看错。

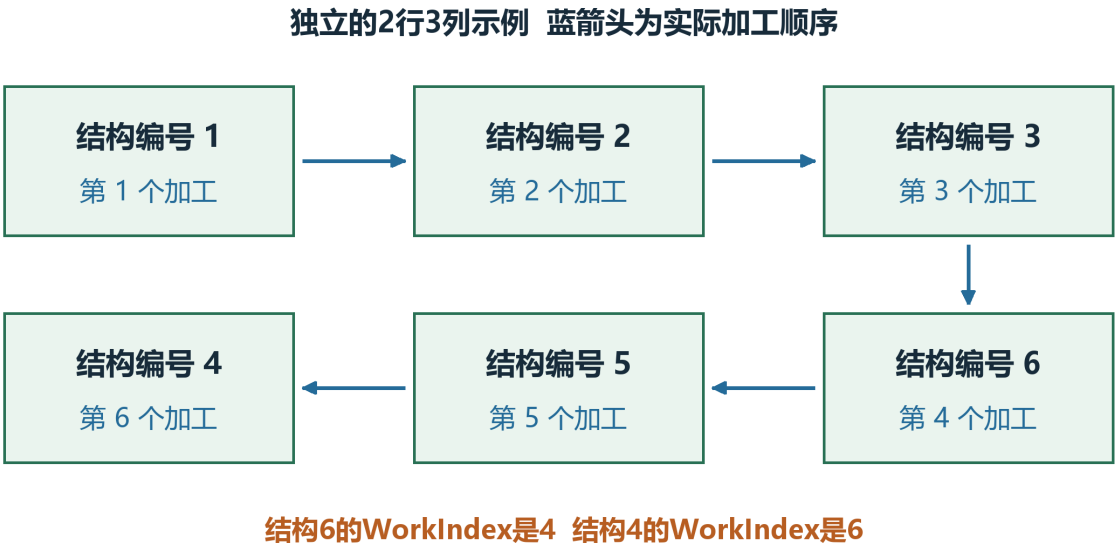


图 4 沿箭头走的是工作顺序 格内固定的是结构编号

另取一个 2 行 3 列的岛阵列。结构编号上排 1、2、3，下排 4、5、6。蛇形加工走 1→2→3→6→5→4，所以第 4 个加工的是结构编号 6。这个小例子只用于讲编号。

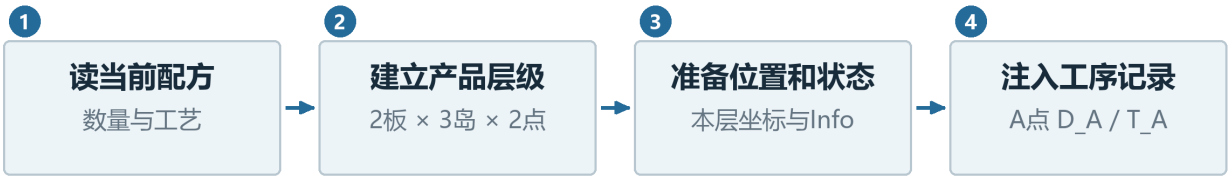
TU 的按序号查询使用 Index；生产消息找基板和基岛时使用 WorkIndex。固晶侧还可能重排列表，Refresh 又沿现有列表赋序号，所以列表第 4 项也不能长期当成同一个物理位置的身份。

输入4看接口	Index 4选结构4	WorkIndex 4选结构6

读到数字 4，先看它表示结构编号还是加工顺序。

05 来料后软件怎样建立加工记录

上料或物料搜索可能先建一份临时 TU。正常启用点胶的入口看到 IsProduct 为 false，会按 S1 参数新建生产用 TU，并把这个标记设成 true。把新对象叫 N1，后面就一直追踪它。



IsProduct只参与生产入口是否重建的判断；它为true不等于产品已加工完成。

图 5 从配方展开 12 个焊点 再给每个点准备工序记录

创建时先建层级和坐标，再按配方设置参与范围，并给焊点补上工序键。例子里 A 点有 D_A 表示点胶、T_A 表示固晶，初始都为未完成；B 点使用 D_B 和 T_B。

有参构造会准备这些数据，还可能整理数量和坐标配置，例如非多件模式会改成 1 件。初始化先注入 S2 再注入 S1 只是程序准备顺序，设备仍按实际生产流程加工。

IsProduct 只参与生产入口是否重建的判断；它为 true 不等于产品已加工完成。

06 设备怎么知道 A 点在哪里

先用没有旋转的小例子看位置。TU 在机器参考位置(100,200)，基板 1 在 TU 内偏(20,0)，基岛 2 在基板内偏(10,5)，A 点在岛内偏(10,5)，单位都为 mm。

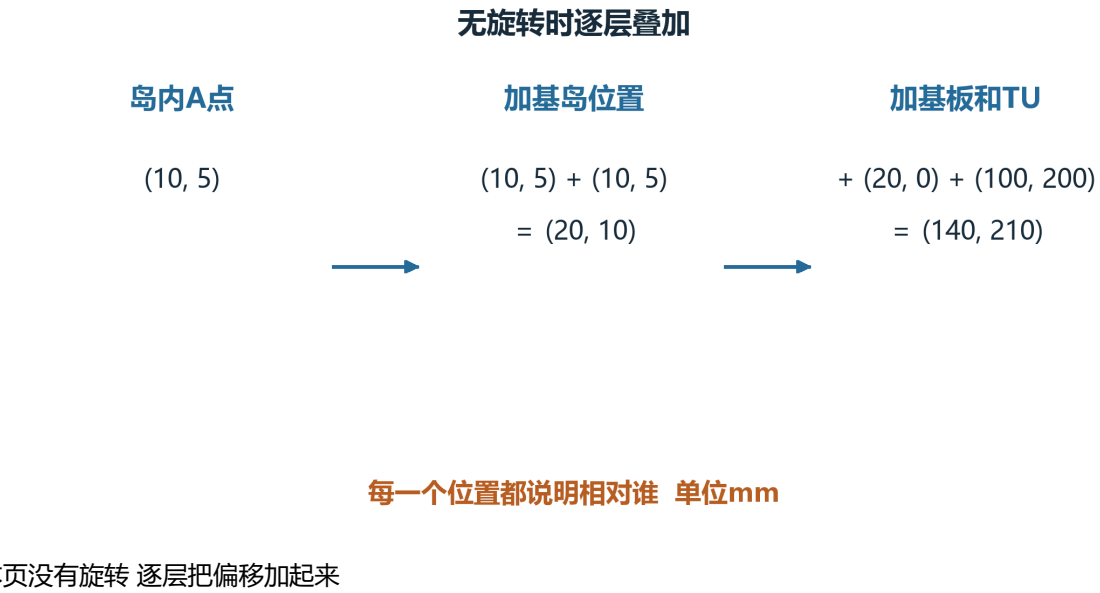


图 6 本页没有旋转 逐层把偏移加起来

A 点先在 TU 内得到(40,10)，再加 TU 位置，得到机器全局参考 G0 中的(140,210)。“在岛内”和“在机器里”是两种说法；软件保存前者，再沿上层关系算后者。

G0 是机器使用的一套全局参考名字。真正发给运动轴之前还要经过设备的坐标转换，所以 G0 数值也不能直接等同于轴显示值。取焊点自身原点时，转换输入是(0,0,0)，不能把配置偏移再加一遍。

位置后面补一句“相对谁”，通常就容易看懂。

07 产品转了方向 A 点会到哪里

沿用上一页的所有长度，只把 TU 转 90°。A 点仍在 TU 内的(40,10)，但 TU 自己的 X 方向已经朝机器的 Y 方向，原来的 Y 方向朝机器的负 X 方向。

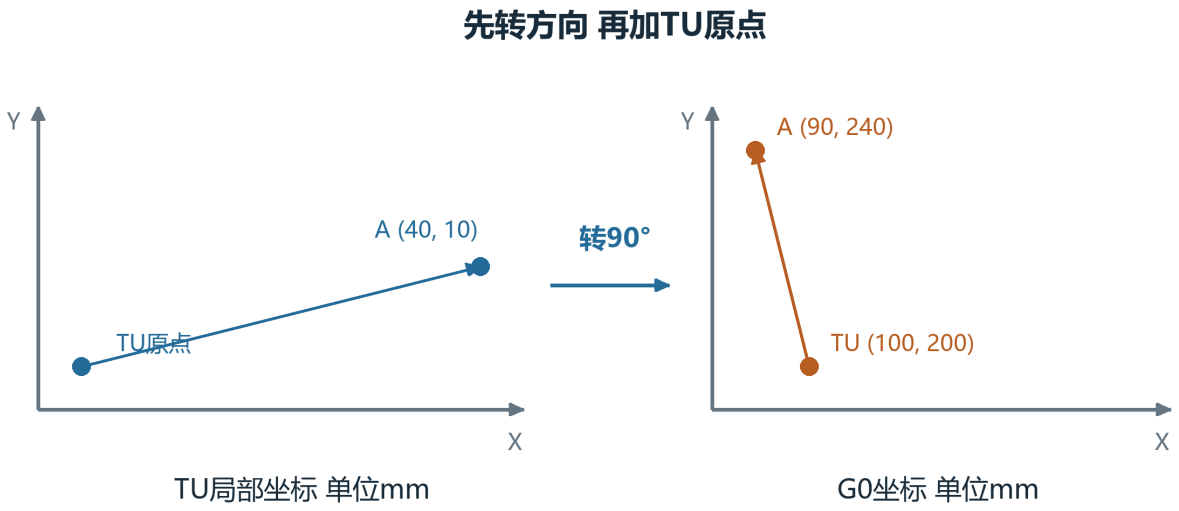
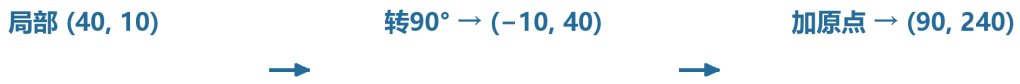


图 7 同一个局部点 随整份产品转到新的机器位置

先转方向：(40,10)变成(−10,40)。再加 TU 位置(100,200)，A 点就在 G0 的(90,240)。后面的贯穿例都使用这组转 90°的输入，下层角度均为 0。

如果基板整体定位后挪了位置，它下面各岛、各焊点在下一次换算时都会跟着变。软件不必逐点改写 12 个焊点的局部配置。程序里的 Degree 使用弧度，90°对应 π/2；界面或转轴使用度时需要转换。



上层的位置和方向变了，下层的机器位置会一起变。

08 点胶工位还有一份整体偏移

同一款产品到了不同工位，可能还要使用本工位的偏移。本例把 S1 偏移设成(2,−1,0)mm。它直接加到 TU 的平移上，整份产品的位置都受到影响。

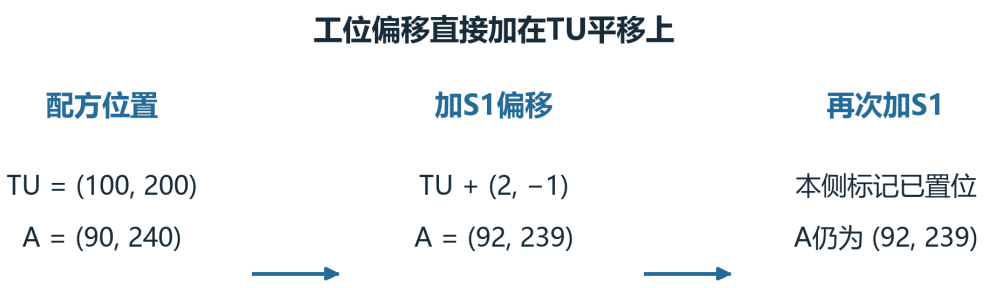


图 8 A 点从 G0 的 90 240 变到 92 239

TU 原来在(100,200)，加 S1 偏移后是(102,199)，所以 A 点从(90,240)变成(92,239)。再调用一次 S1 偏移方法，标记已置位，数值保持不变。

S1 和 S2 各有独立的防重复标记；它们不能互相替代。Refresh 会重新从配置建坐标并清掉这两项标记，却不会自动再加本侧偏移。字符串方式创建 TU，也不会自动选 S1 或 S2 偏移。

是否带工位偏移，要看最后一次创建、偏移或刷新发生在什么时候。

09 拍照定位和测高改了什么

点胶前，可以先通过相机找产品特征，再测实际高度。定位改变的是被定位那一层的当前位置和方向；测高把“实测平均高度减示教平均高度”的差，加到这一层的 Z 上。

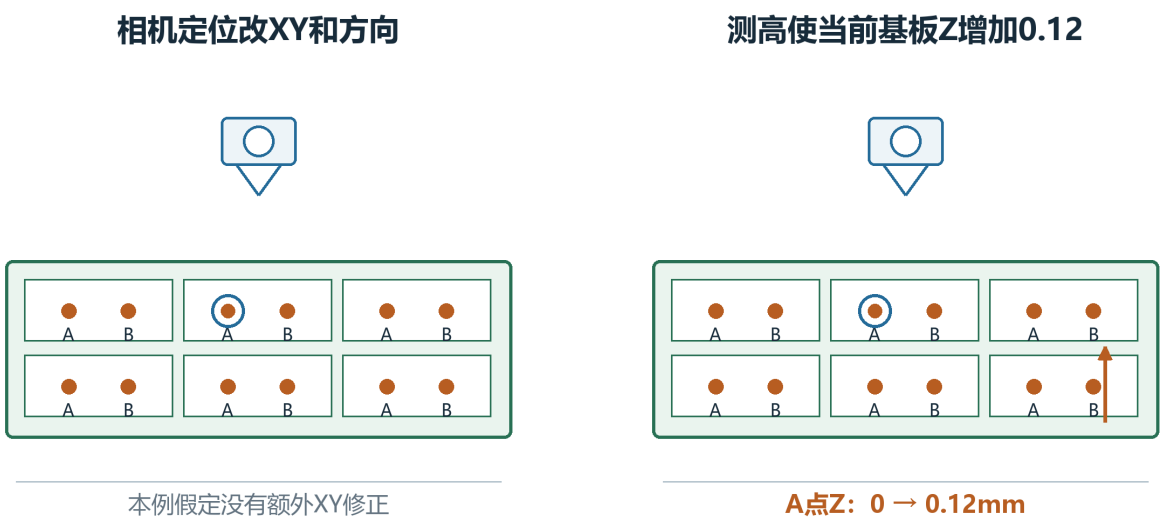


图 9 先看 XY 定位 再看基板抬高对下层 A 点的影响

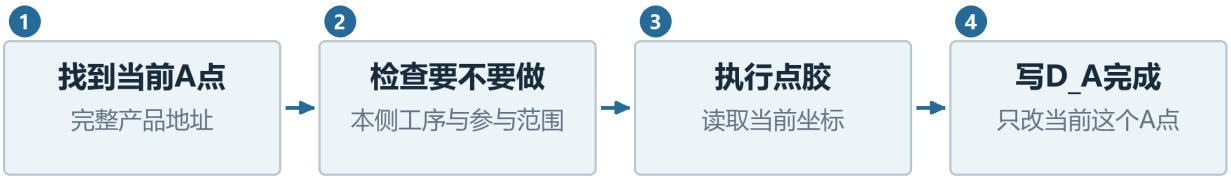
本例先假定 XY 没有额外定位修正，只给基板 1 测到+0.12mm。A 点的 G0 位置从(92,239,0)变成(92,239,0.12)。同一基板下面的其他点也会带上这个高度变化。

结果数值保存在 Info 里，坐标另外更新；IsVisioned、IsMeasureHeight 是处理标记。定位关闭的分支也能把定位标记置 true，所以只看一个 true，不能认定真的拍过照片。纠偏册会继续讲拍照结果怎样变成位置。

看本次结果时，把数值、处理标记和当前坐标放在一起。

10 A 点点完胶后记录写到哪里

动作控制器拿到一条“给这个 A 点执行 D_A”的工作消息，按工作序号找基板和基岛，再按焊点名找 A 点。动作先检查参与范围和当前工序，再读取位置并执行点胶。



“当前动作走完”和“当前焊点这一项工序完成”，由两处记录分别表达。

图 10 点胶完成后只改变当前 A 点的 D_A 记录

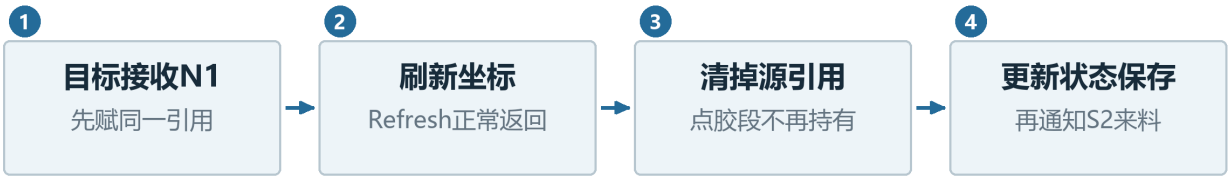
点胶动作成功后，把基板 1、基岛 2、A 点自己的 D_A 改成 true。T_A 仍为 false，其他 11 个焊点各自保留原来的进度。一个岛有 A、B 两个点，也能有不同工序配置。

工作消息的 Finished 由控制器另外标记。后面取下一条消息继续执行；返回 Retry 会按任务逻辑重试，其他非成功结果进入相应停止处理。工序注入只补缺少的键，不会自动把已有 true 改回 false。

“当前动作走完”和“当前焊点这一项工序完成”，由两处记录分别表达。

11 点胶完成后怎样交给固晶工位

点胶侧允许放行后，输送任务还要确认源段有料、目标段无料。机械移送和检测完成，才进入下面的软件交接；输送任务取得成功结果后，再通知固晶侧来料完成。



放行、实物移动、软件交接、来料通知是不同节点。

图 11 按 1 到 4 看软件交接 N1 仍是原来那份产品

先把点胶段的 N1 引用交给固晶段，再对 N1 执行 Refresh。刷新正常返回以后，才清掉点胶段引用，更新区段状态并保存。正常成功出口是点胶段无料、固晶段有料。

N1 连同原来的基板、岛和焊点继续被使用。交接传的是同一份对象入口，并没有把进度复制进另一棵新树。源、目标还分别记录移送阶段，如 Ready、Transfering、Alarm，不能和有料状态混在一起看。



放行、实物移动、软件交接、来料通知是不同节点。

12 为什么点过胶却还需要重新定位

答案在 Refresh 这一步。可以把它读成“按当前配置重新准备这份产品的坐标”。它保留 N1 和焊点对象，但把各层坐标对象重新建立，清除定位、测高和工位偏移标记。

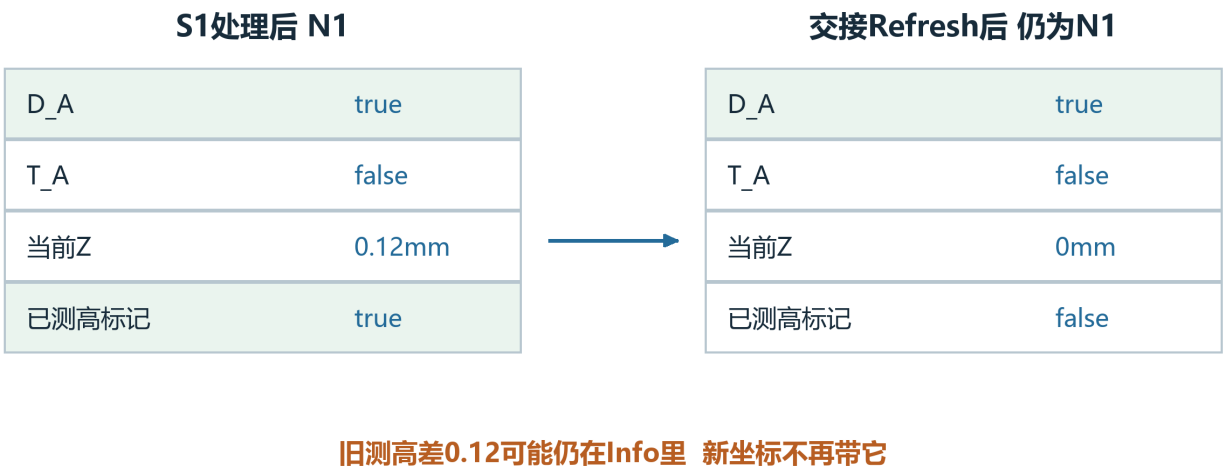


图 12 左侧交接前 右侧交接后 工序保留而位置重建

本例 A 点在 S1 测高后是(92,239,0.12)。交接刷新后回到配方链给出的(90,240,0)。D_A 仍为 true，T_A 仍为 false；旧测高差 0.12 还可能读得到，但当前坐标已经不包含那次增量。

身份识别标记、识别代码和 IsProduct 也保留。Refresh 不按新数量重建结构，不重新应用参与配置，也不重新注入工序。若代码提前保存过旧坐标引用，刷新后应从当前产品对象重新取得坐标。



进度保留，位置重建；两件事可以同时发生。

13 到固晶工位一定会新建 TU 吗

正常 S1 已经把 N1 的 IsProduct 设成 true，Refresh 又保留了它。启用固晶、已配置输送时，S2 只在这个标记为 false 时按 S2 参数重建，所以正常双工位路径通常继续使用 N1。

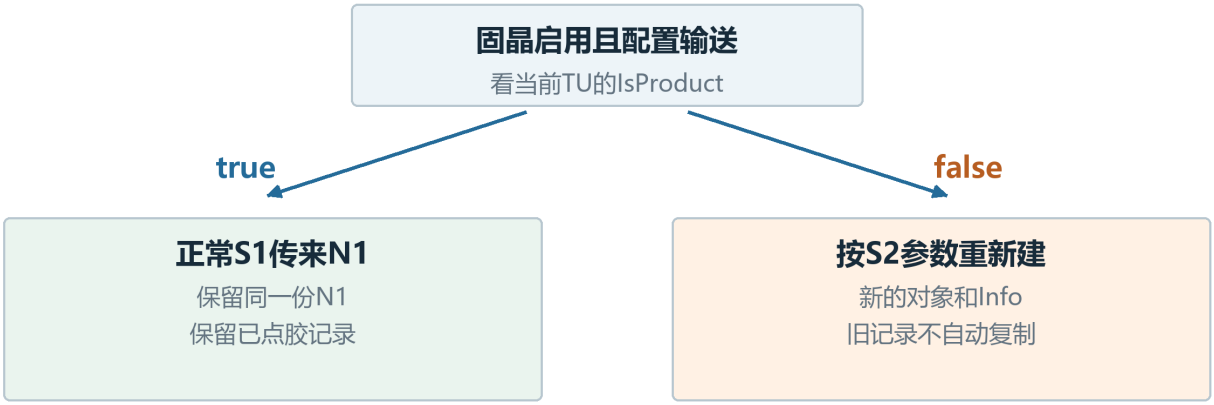


图 13 先看 IsProduct 再看是否满足 S2 创建条件

若点胶任务的 Enable 为 false，它可能跳过设置标记的生产分支再放行。S2 取得的 TU 仍为 false，满足上述条件时就会重新创建，新 Info 不会继承旧加工记录。两种情况要分开看。

本册贯穿例把 S2 工位偏移设为 0，并假定后续没有额外定位和补偿。因此保留 N1 进入 S2 后，A 点暂时仍是(90,240,0)。保留路径没有执行 S2 新建，也就不能凭想当然再加一次 S2 构造偏移。

追踪同一块产品时，既看产品名称，也看实际对象是否被替换。

14 固晶完成后为什么有两套完成记录

固晶动作成功，会把 A 点的 T_A 改成 true。此时同一个 A 点是 D_A=true、T_A=true。控制器还有本轮拍照、取片、纠偏、贴装等消息，各自记录是否走完。

产品A点的工序

D_A	true
T_A	true
记录归属	当前N1

控制器的工作消息

取片消息	完成
纠偏消息	完成
贴装消息	完成

Finish不清产品工序

Finish复位消息供后续使用

消息和产品记录是两本不同的账

图 14 产品的工序记录和控制器的动作消息分别更新

控制器 Finish 把工作消息准备成下一轮可用的状态，不清掉 TU 上的 D_A、T_A。只重置某条消息，也不会自动清掉焊点 Info 里保存的旧偏移或识别结果。

一个工序是否需要执行，还要结合参与范围、EntityState 及对应字典值。EntityState 的 Process、Success、Fail 是另一项状态；把 T_A 写成 true 不会同步把它改成 Success，也不能单凭这项记录证明实物质量合格。

查“为什么又做了”或“为什么没做”，要明确查的是动作消息还是产品工序。

15 产品做完以后怎样离开设备

配置输送时，固晶侧正常发送会发出允许移向待下料段的信号。后续区段按自己的机械条件接力，直到产品离开。软件的当前 TU 入口也随这些成功路径交接或清除。



图 15 不同下料方式 对 TU 的处理并不完全一样

固晶到待下料，先给目标引用并置有料，再按条件刷新，最后清源。待下料到皮带会交接引用并刷新；到料仓的路径清掉等待段 TU，没有把旧树赋给下料段的同样代码。

未配置输送的固晶路径，可按配置导出 Mapping，再清固晶段 TU、置无料，提示人工取走并停止。所以下料看到哪个区段还持有 TU，要对照实际走的入口。

软件清了当前引用，不等于实物一定已被人工拿走。

16 分两段点胶时产品记录怎么跟着走

长产品可能一次够不到全部位置，需要移到第二段继续点胶。Divide 模式把消息分成两组，中间插入一个等待移段的动作。两组由配置索引集合划分，数量不一定相等。

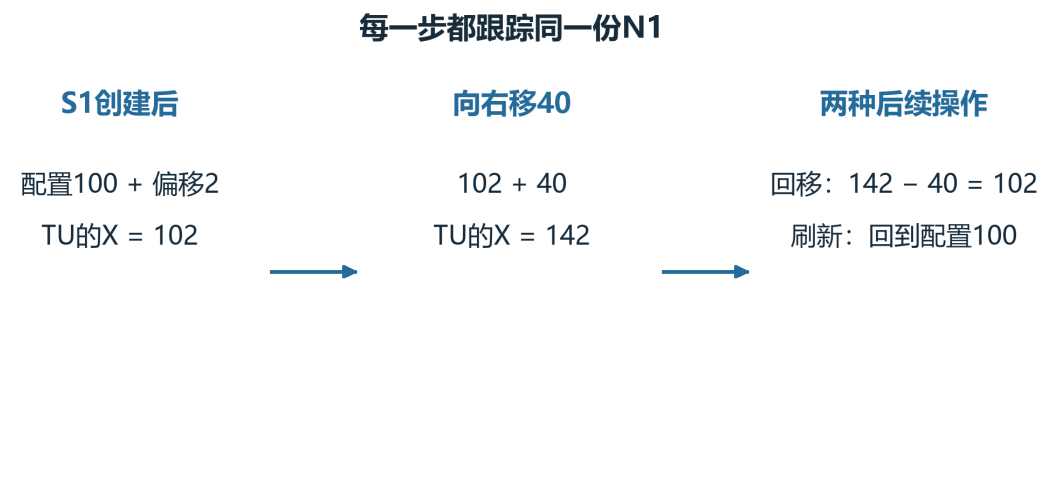


图 16 同一 N1 从第一段移到第二段 只给 TU 的 X 加距离

假设配方 TU 的 X 为 100mm，S1 偏移+2mm，移段距离 40mm。S1 创建后是 102，移段后是 142；若搜索走回移分支，减 40 后回到 102。若走 Refresh，结果是配置值 100，含义完全不同。

正常移段保留对象和工序，不统一清定位、测高标记。第二组是否再拍照，看各层自己的条件；TU 级定位在 Divide 分支直接跳过。InteractiveAction 还要等完成信号后才能继续第二组。

移段是改当前位置；刷新是重建坐标，不能互相代替。

17 停用一个点和加工完成有什么区别

参与范围回答“这个点要不要经过这一侧”。Enable 两侧都参加，只点胶和只固晶各有选项，Disable 两侧都跳过。完成记录则回答“这个点的这道工序做过没有”。



图 17 同样是无需再做 可能因为完成也可能因为不参与

基板和基岛会汇总下面焊点的参与范围；TU 根的参与属性没有采用同样的汇总方式。工序完成则逐层看下层是否完成，空集合也可能返回 true，所以“已完成”不能证明结构齐全。

当前 S1 完成函数有个特别条件：S1 非空、S2 为空时，即使所有 S1 项已完成，它仍返回 false。S2 字典为空时则返回 true。排查“明明点完了却未完成”时，需要同时看两侧工序配置。

停用、工序完成、处理成功和质量合格是不同信息。

18 在生产 Mapping 里点正常或成功会怎样

生产 Mapping 显示的是区段当前产品。允许修改时，选择一个焊点再点“正常”，会把这个点的两侧工序恢复成未完成；点“成功”会把两侧工序都置成完成。

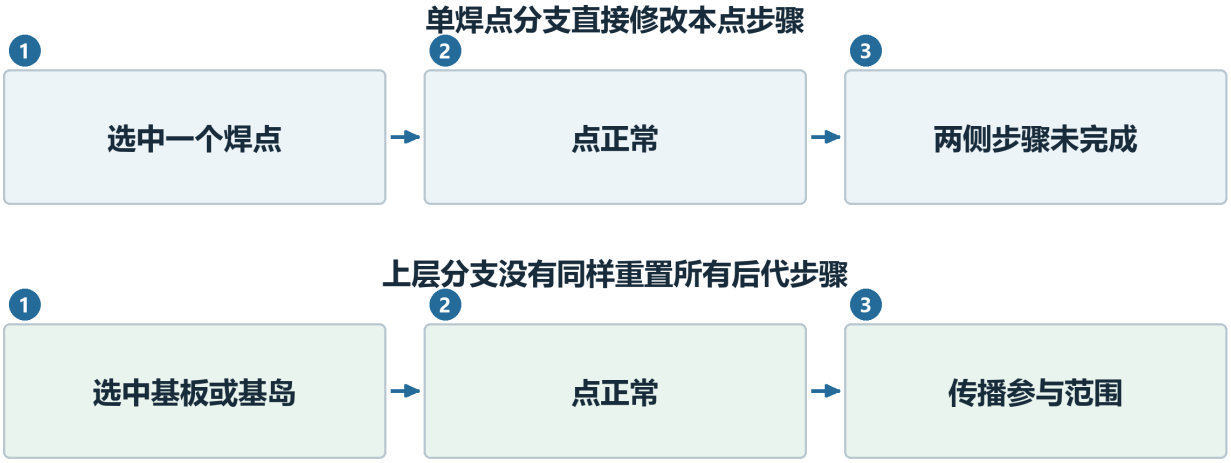


图 18 选中焊点和选中上层区域 按钮影响不同

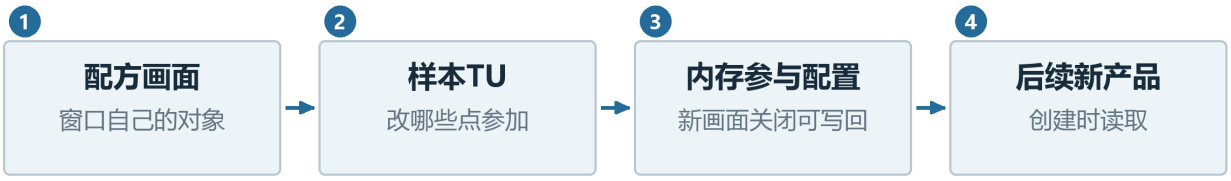
停用一个焊点也会重置它的两侧步骤。若选中的是基板或基岛，按钮主要向下传播参与状态，这条分支没有同时按单焊点方式重置所有后代字典。

所以把整块基板设为正常，不一定就把每个焊点都变成待重做。人工把 A 点置成功，也只是改了软件记录，不能说明机器真的完成过这一点的点胶和固晶。相关按钮还检查机器工作状态。

先看选中的是一个焊点，还是一整层区域。

19 配方 Mapping 改的是后续产品的规则

配方 Mapping 画面很像生产 Mapping，但它先建立一份窗口样本 TU。改样本上的参与范围，再写回配方相关数据，让后续创建的产品读取。它没有保存样本的完整加工进度。



画面改过、内存改过和已保存到文件，要分开确认。

图 19 配方画面和生产画面 最终修改的数据拥有者不同

新配方 Mapping 关闭且有变化时，SaveInformation 只把参与范围写回内存；关闭事件和这个方法没有调用配方 Save。旧画面关闭会调用保存回调。工作方向、蛇形和新画面清空按钮又有自己的保存路径。

关闭配方窗口后，如果检测到某区段还有 TU，主界面可能进入清空确认；仍要满足机器状态检查并取得操作者确认才执行。留下旧 TU，不表示它自动套上新参与配置。

画面改过、内存改过和已保存到文件，要分开确认。

20 搜索物料会不会找回原来的进度

物料搜索是把传感器、人工确认和区段数据重新对上。若原 TU 仍在，某些分支可以继续用它；若原 TU 已经没了，确认有料后新建的是当前配方下的新记录。

S1和已配置输送S2的主要分支

检测有料	TU存在	保留并Refresh
检测有料	TU为空	确认有料才新建
检测无料	TU存在	确认有料则保留
检测无料	TU存在	确认无料则清空
检测无料	TU为空	保持空引用

保留原对象才能保留旧Info和工序

图 20 检测结果和当前 TU 一起决定保留 新建或清空

以 S1 和已配置输送的 S2 为例：检测有料且 TU 存在，保留并 Refresh；检测有料而 TU 为空，人工确认有料才新建。检测无料却已有 TU，确认有料则保留且不走 Refresh，确认无料则清引用。

自动工作、不反搜且已有 TU 时还可能提前返回。S2 未配输送改用真空检测，有自己的重试分支；待下料和下料的某些“有料但 TU 空”分支不会补树。单看 HasMaterial 无法证明 TU 存在。

新建电子记录不会自动找回丢失的旧加工进度。

21 清空载具到底清了哪些东西

整线清空会把五个区段的当前 TU 引用设为空，并把有料状态置 NoMaterial。只清点胶或只清固晶，则只处理选定区段。操作前有机器状态检查和确认。

	整线清空	只清点胶	只清固晶
上料	●	○	○
点胶	●	●	○
固晶	●	○	●
待下料	●	○	○
下料	●	○	○

清引用并置无料　不统一重置全部信号和消息

图 21 橙色表示解除当前区段持有的 TU 入口

这几个控制器方法在 Working 或 Pause 时直接返回，快捷菜单还另查 IsStop。正常分支会执行对应安全移动和松夹等操作，再按其流程保存；不能只凭按钮按过就推断清空已经发生。

清空没有逐节点删除旧 Info，也没有统一复位移送状态、信号和动作消息。某个窗口先前拿到旧 TU 引用，仍可能读到旧对象；判断当前产品时应重新从区段获取。

刷新：仍是N1	新建：得到新记录	清空：区段入口为空

清空是让区段不再持有那份 TU；刷新仍保留同一份 TU。

22 换了数量或产品形状为什么要重建

只调位置且结构仍匹配时，可以刷新坐标；把每板 3 个岛改成 4 个，旧 TU 不会靠 Refresh 自动多出一个岛。改焊点名、增删层级或替换异形节点时，需要重新建立对应运行树。

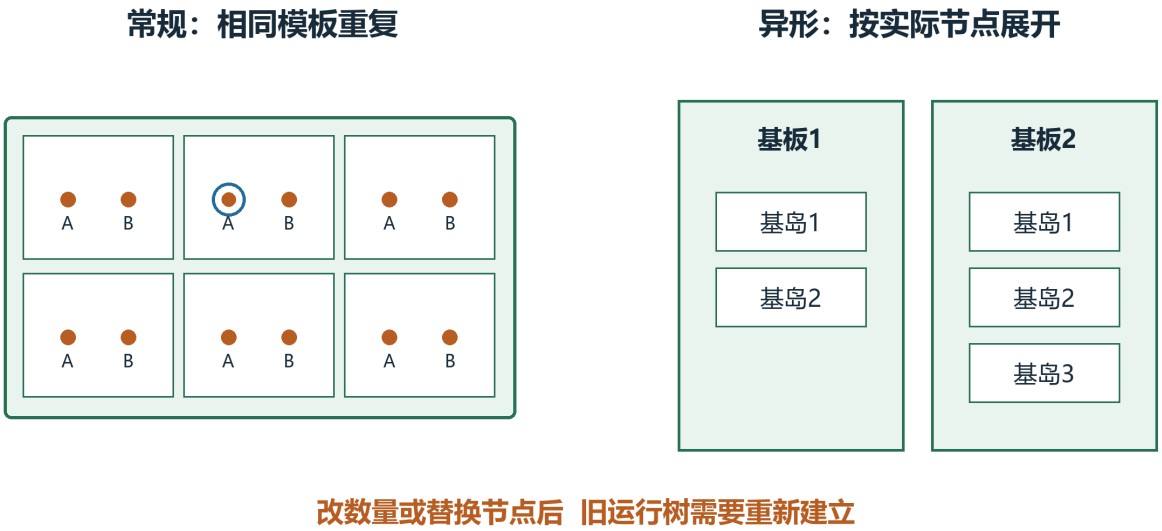


图 22 规则产品重复展开 异形产品按实际节点展开

异形产品可以一块基板有 2 个岛，另一块有 3 个。节点位置描述摆放，ConfigName 决定读哪份工艺配置，所以多个位置仍可以共用一份 A 点工艺。数量要沿实际树统计。

焊点的镜像标记是在创建时复制的，Refresh 不会重新赋值。工序改名后再注入，会补新键却留下旧键。常规对象又动态读取当前配方，因此树、配方、工序和镜像要一起核对。

改变产品结构以后，保存配方和重画页面都不会自动更新旧运行树。

23 异形编辑中的名字相同为什么还找不到

异形编辑器操作的是配方节点。新增、删除、复制会改变这棵配置树；运行 TU 仍关联创建时的节点。编辑控件重新排序 ID、重画页面，也没有替旧 TU 重新建后代。

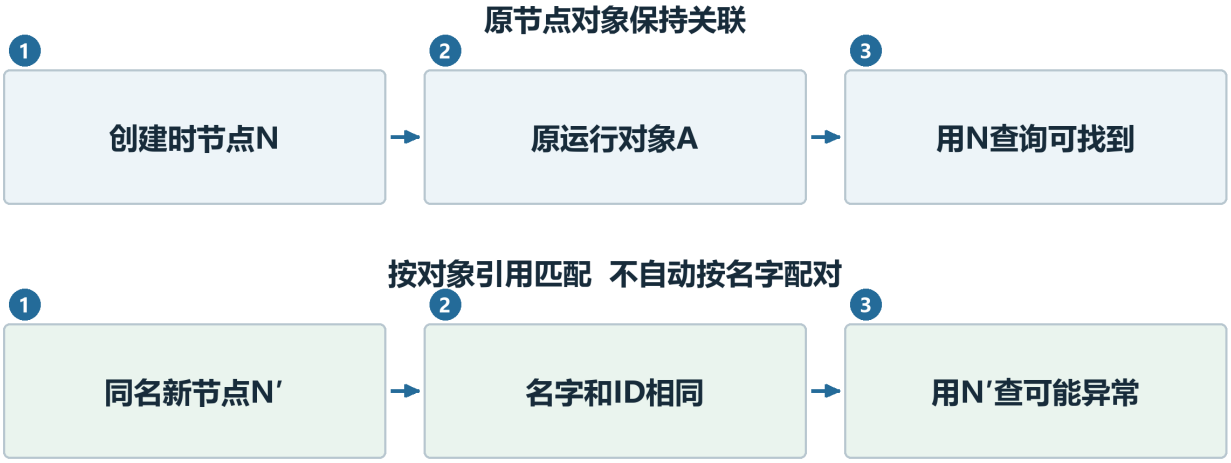


图 23 同一个节点对象能关联 另建的同名节点不一定能关联

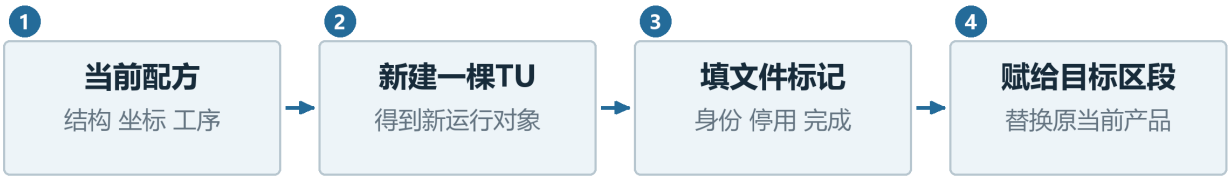
按节点反查运行对象时，GetBaseMatterByOppositeSex 比较的是节点对象引用。另建一个名称、ID、ConfigName 都相同的节点，仍可能找不到并抛异常。它不按名字自动配对。

复制节点时，位置和显示名可以变化，ConfigName 仍可能沿用。删除最后一个引用某 ConfigName 的节点，还会删除对应公共配置及 Mark PR；读取缺失配置又可能建立默认项并保存。这些都要结合实际节点树看。

界面名字、节点 ID、运行编号和对象引用，各有自己的用途。

24 导入 Mapping 为什么不能等于原样续作

Mapping 导入先按当前配方新建一棵 TU，再把文件中的身份和结果标记填进去。结构、坐标和工序来自现在的配方，文件没有带回原来完整的运行对象和实时位置。



导入结果可用来显示或恢复指定标记，不能据此保证断电后的完整续作。

图 24 当前配方提供新树 文件只补回规定的信息

标准路径里 X 表示停用，2 让对应焊点两侧工序都完成，1 保留新建树的初始步骤。芯片代码路径还会对非 X、非 1 内容写 ComponentCode 并置完成。布局和配方需匹配，异常文件可能读取失败。

导出 Mapping 用于表达产品结果和身份信息，不包括所有 Info、机器父坐标和动作消息。显示已有 TU 的界面还可能把 IsProduct 设成 true；这个标记依然不能证明文件对应的实物已完成加工。

导入结果可用来显示或恢复指定标记，不能据此保证断电后的完整续作。

25 保存过配置重启就能继续吗

正常交接后有 Save 调用，但当前区段 TU 和坐标等数据被排除在配置序列化之外。保存配置，没有同时保存一份包含完整产品树、父坐标链接、工序和控制器消息的加工现场。

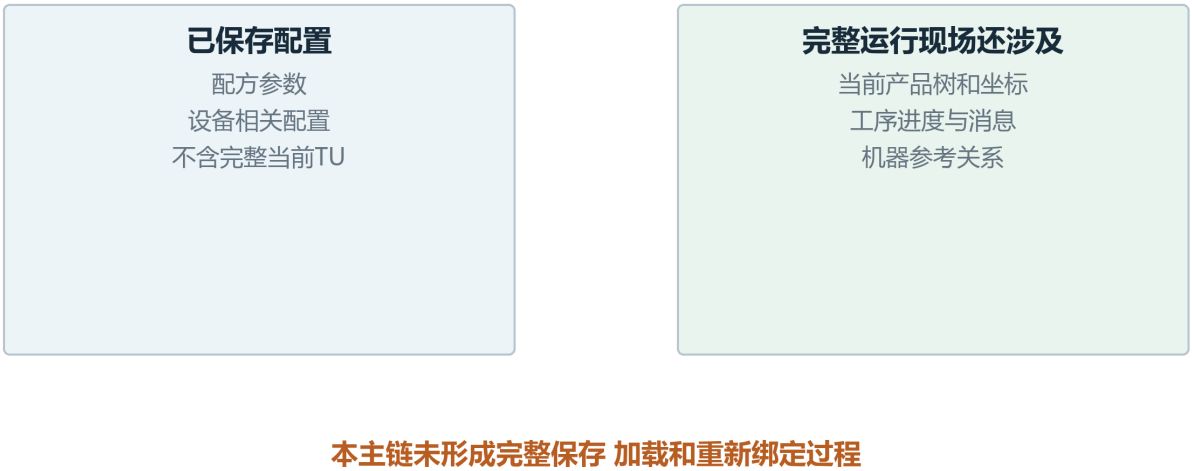


图 25 配置文件和完整运行现场 包含的内容不同

项目也有独立 TU 存储接口，不过在本册追踪的正常生产主链里，没有形成完整的保存、加载和重新绑定过程。文件存在、Save 返回或 Mapping 导出，都不能单独证明具备完整恢复能力。

重新启动时，物料搜索和刷新按各自分支处理软件数据；需要让当前区段对象、检测结果和实物位置相互对应。这里讲现有实现的能力，不把补建记录描述成恢复全部历史。

要解释恢复了什么，先看实际保存了什么。

26 交接报错时产品可能已经走了一半

程序报 Exception，并不表示一切还停留在调用前。点胶到固晶的目标引用赋值在 Refresh 之前；刷新若中途抛异常，源和目标可能暂时同时指向 N1，部分坐标也可能已重建。



图 26 竖线标出失败位置 其左侧修改可能已经发生

若错误更晚，发生在清源和状态更新后的 Save，内存可能已完成交接，源引用也已为空。卡料选择 Retry 回机械重试段，不回滚物理运动；Ignore 还可能继续到成功出口。

其他输送入口的次序又有差别，部分 catch 只恢复一个有料状态，没有恢复全部引用和坐标。因此判读时要同时看实物在哪、两区段数据、失败行和本次来料信号。

Refresh失败：可两边同指N1	Save失败：源可能已清空

失败返回说明走了哪个出口；已经发生哪些变化，要看失败之前的步骤。

27 示教和测试用的 TU 是哪一份

示教、移动到焊点窗口和辅助测试也会创建 TU。有的赋给生产区段，有的只留在窗口或局部变量里。窗口出现一份新产品图，不一定代表区段当前 TU 也被换掉。



图 27 先看对象由谁持有 再看新建是否影响生产区段

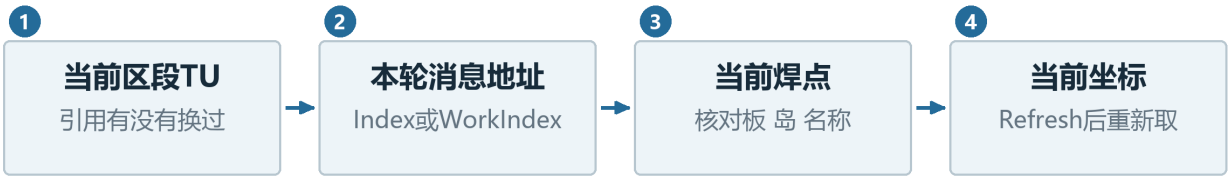
在线 JudgeTuExist 可经确认重建已有区段 TU；离线返回窗口新 TU，不赋给区段。补偿工作模式会先建“测试”TU；固晶启用且配置输送时还可能再按 S2 重建一次，前一份测试记录不复制。

角度示教又是分阶段写入：窗口先改传入坐标，调用方后面再复制到其他层或配置。基板入口传入的可能是 TU 坐标；后续 PR 取消也没有统一撤销前面写入。具体两点过程见纠偏册。

确认是生产产品、窗口样本还是测试对象，再解释它的位置和记录。

28 开发时按这个顺序查到真正的 A 点

从当前区段取得 TU，再确认本轮消息的结构或工作编号，最后找焊点名。不要长期缓存一个“当前 A 点”，因为新建会替换 TU，Refresh 会替换坐标，而解析未命中还可能留下旧缓存。



源码入口和其余实现细节放在本版配套阅读索引，按问题查阅即可。

图 28 当前区段 当前消息 当前对象 当前坐标要对应起来

按 Index 可调用 GetSubstrate、GetModule、GetBondPosition；未找到可能返回 null，参数错误也可能抛异常。按工序查询之前要先注入步骤，直接读缺失键会抛异常。名称相近的位置服务也可能返回不同参考系。

当前 GetAllBondPositions 按名称的循环会漏掉常规最后一板和最后一岛。本例按 A 名本应 6 个却只得 2 个；GetAllBaseMatter 则返回 20 个后代，不含 TU 根。异形数量也不能套常规 Count 或每板最大值来当总数。

源码入口和其余实现细节放在本版配套阅读索引，按问题查阅即可。

29 把同一个 A 点从来料追到完成

现在把整册连起来。说明输入固定为 2 板、每板 3 岛、每岛 A/B，TU 位置(100,200)并转 90°；S1 偏移(2,−1)，基板 1 测高+0.12mm，S2 偏移 0，其余定位和贴装补偿在本例设 0。

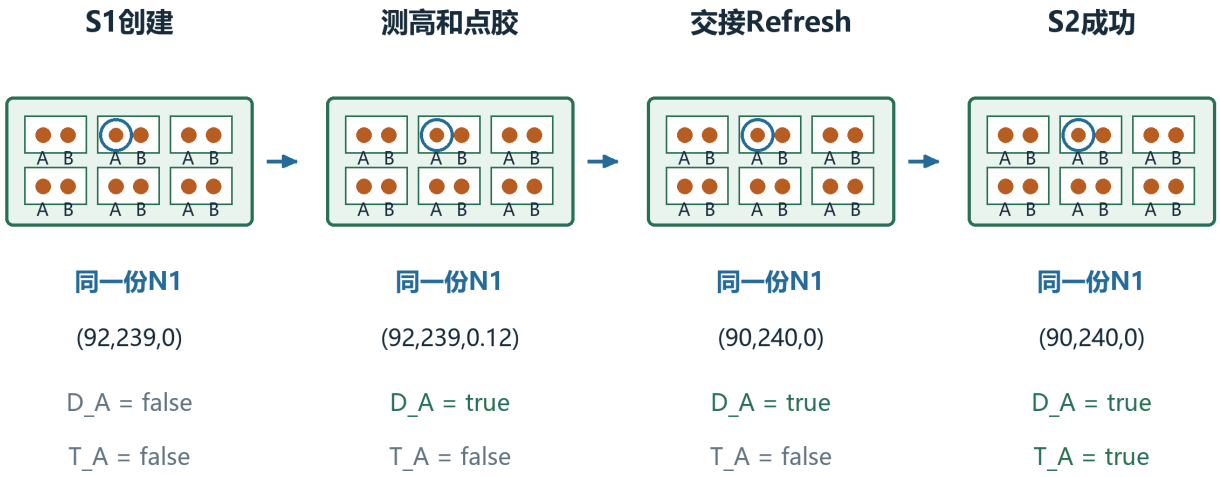


图 29 每列跟踪同一个 N1 和同一个 A 点 位置单位 mm

S1 创建 N1 后，A 点在(92,239,0)，D_A 和 T_A 都为 false。测高后 Z 变 0.12；点胶成功只让 D_A=true。交接 Refresh 仍用 N1，位置回到(90,240,0)，D_A 保留。

S2 看到 IsProduct=true，正常保留 N1；成功完成这个 A 点的 T_A 后，两项才都为 true。其他 11 个点各查自己的记录。实际贴装还要叠加芯片及工具补偿，不能把这页 A 点原点直接当作完整运动命令。



这份记录帮软件记住哪一块、哪一点、做到哪；当前坐标帮动作找到它现在的位置。